

IN3370

Ana María Jiménez Güiza

September 2024

1 Prosessering av portrett for ansiktsgjennkjenning

1.1 Mellom-resultat-bildet etter gråtonetransformen.

For å finne koeffisientene i den lineære transformen (a- og b-parametrene) brukte jeg formelen for justering av middelerdi og varians.

Siden vi hadde verdiene av den middelerdi og varians som vi ønsket oss til bildet med høy kontrast, kunne vi regne den nåværende middelerdi og kontrast, og etterpå bruke disse verdiene til å regne koeffisientene.

Først regner vi middelerdien med likningen:

$$\mu = \frac{1}{n \times m} \sum_{x=0}^{n-1} \sum_{y=0}^{m-1} f(x, y) = 106.63173640593243 \approx 106.63$$

Nå bruker vi middelerdien til å regne variansen av originalbildet:

$$\sigma^2 = \frac{1}{n \times m} \sum_{x=0}^{n-1} \sum_{y=0}^{m-1} [f(x, y) - \mu]^2 = 609.3055277487697 \approx 609.31$$

og

$$\sigma = \sqrt{\sigma^2} = 24.6841148868816 \approx 24.68$$

Den verdien som vi ønsket oss for middelerdien er 127, og standardavviket er 64.

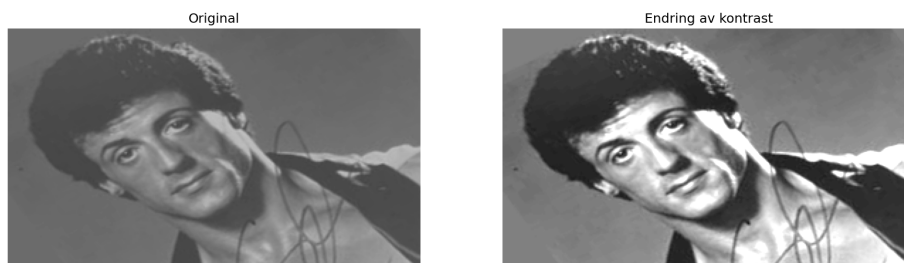
Da har vi at

$$a = \frac{\sigma_T}{\sigma} = 2.592760578748273 \approx 2.59$$

og

$$b = \mu_T - a \cdot \mu = -149.47056259677868 \approx -149.47$$

Siden nå har vi verdiene for a og b , så kan vi bruke disse for å regne de nye verdiene av bildet for å gi det bedre kontrast. Vi kan se forskjellen mellom originalen og etter å endre kontrasten.



1.2 Koeffisientene for affine transformen og baklengstransformen

For å finne koeffisientene for affine transformen valgte jeg fire pikler som mine start punkter, både i den original bildet og i geometrimasken. Disse pikler var øynene og kantene av munnen.

	Portrett	Geometriskmasken
Venstre øye	84, 89	170, 258
Høyre øye	119, 69	340, 258
Venstre hjørne munnen	116, 116	193, 441
Høyre hjørne munnen	144, 100	316, 441

For å finne koeffisientene brukte jeg de punktene i tillegg til formlene

$$x' = a_0 \cdot x + a_1 \cdot y + a_2$$

$$y' = b_0 \cdot x + b_1 \cdot y + b_2$$

Ved å sette x' , y' , x og y for hvert startpunkt så har vi 8 formler.

$$\begin{aligned} 170 &= a_0 \cdot 84 + a_1 \cdot 89 + a_2 \\ 258 &= b_0 \cdot 84 + b_1 \cdot 89 + b_2 \end{aligned}$$

$$\begin{aligned} 193 &= a_0 \cdot 116 + a_1 \cdot 116 + a_2 \\ 441 &= b_0 \cdot 116 + b_1 \cdot 116 + b_2 \end{aligned}$$

$$\begin{aligned} 340 &= a_0 \cdot 119 + a_1 \cdot 69 + a_2 \\ 258 &= b_0 \cdot 119 + b_1 \cdot 69 + b_2 \end{aligned}$$

$$\begin{aligned} 316 &= a_0 \cdot 144 + a_1 \cdot 100 + a_2 \\ 441 &= b_0 \cdot 144 + b_1 \cdot 100 + b_2 \end{aligned}$$

Videre kalkulerte jeg for hver gruppe av tre de tilsvarende koeffisientene, og brukte de for å regne den minste kumulativ feil, og da fant jeg ut at de beste koeffisientene for den affine transformasjonen var:

$$\begin{aligned} a_0 &= 3.022 \\ a_1 &= -3.211 \\ a_2 &= 201.956 \end{aligned}$$

$$\begin{aligned} b_0 &= 2.309 \\ b_1 &= 4.041 \\ b_2 &= -295.618 \end{aligned}$$

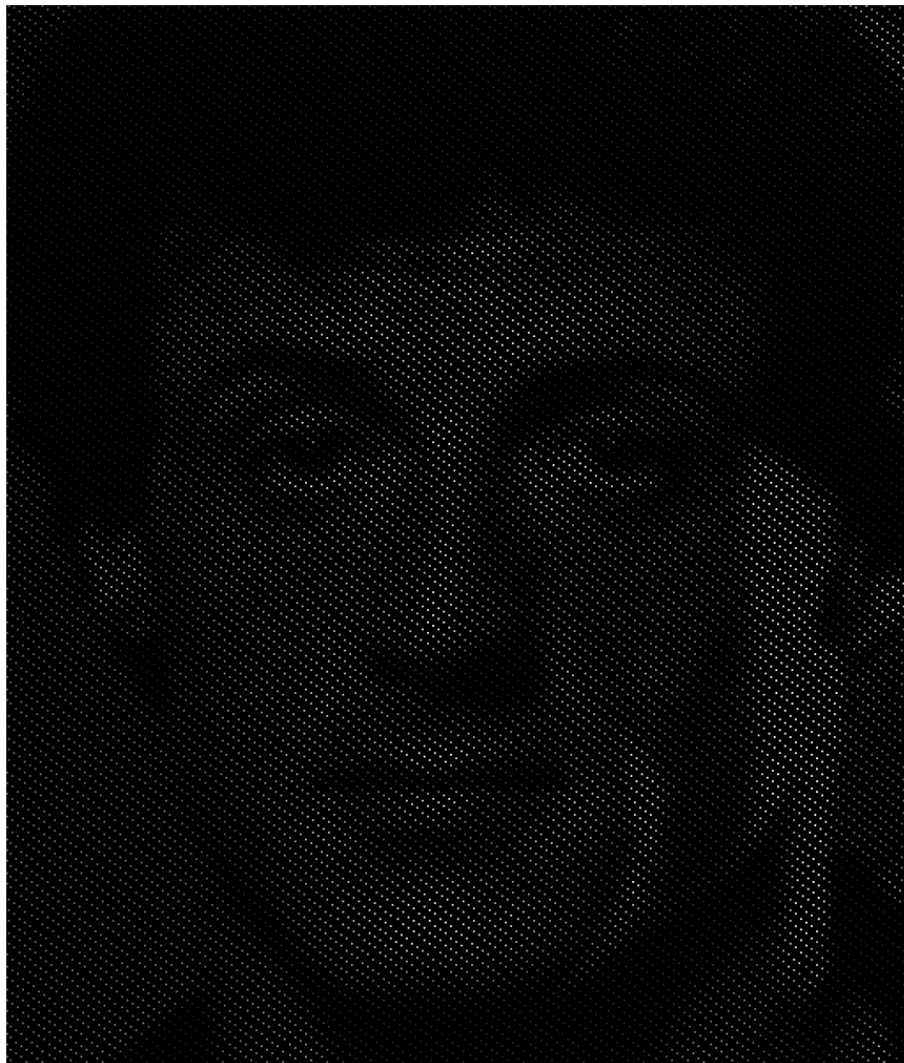
Jeg skrev også disse koeffisientene som en matrise C , slik at

$$C = \begin{bmatrix} a_0 & a_1 & a_2 \\ b_0 & b_1 & b_2 \\ 0 & 0 & 1 \end{bmatrix}$$

For å finne koeffisientene for baklengstransformasjoner så brukte jeg numpy funksjonen for å finne inverse av en funksjon.

1.3 Resultatene av forlengs- og baklengstransformasjoner

Portrett etter forlengstransformasjon



Portrett etter nabointerpolasjon



Portrett etter bilineær interpolasjon

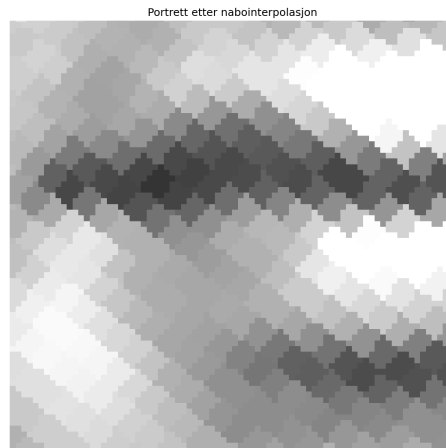


1.4 Forskjeller mellom resultatene.

Som man kan se det er stor forskjell mellom forlengs- og baklengstransformasjonene. Resultatet fra forlengstransformasjonen ser ikke veldig bra ut, og dette er på grunn av det er mange "hull" i bildet. Det vil si, at det er mange pikler som ble ikke farget siden det var ikke en presis korrespondanse mellom piklene fra den original bildet til den geometriskmaske.

På den andre side, begge baklengsmetodene ga veldig gode resultater. Bilinear interpolasjon gir en mer smooth resultat enn nabointerpolasjon. Når man ser nærmere til piklene på nabointerpolasjon er det veldig interessant å se at i

de baklengsmetodene det virker at piklene har også endre form pga. transformasjonen.



1.5 Koden

```
1 from PIL import Image
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # DEL 1
6 # Standardisere kontrasten slik at
7 # - middelvei er på rundt 127
8 # - standardavvik på rundt 64
9
10 filename = 'portrett.png'
11 img = Image.open(filename).convert('L') # 'L' mode ensures the image is
    ↳ grayscale
12 img = np.array(img)
13 n, m = img.shape
14
15 # Regner middelveien
16 def mid_verdi_regning(n, m, img):
17     temp = 0
18     for i in range(0, n-1):
19         for j in range(0, m-1):
20             temp += img[i,j]/(n*m)
21     return temp
22
23 mid_verdi = mid_verdi_regning(n,m,img)
24
25 # Regner varians og standardavvik
26 def varians_regning(n,m,img, mid_verdi):
27     temp = 0
```

```

28     for i in range(0, n-1):
29         for j in range(0, m-1):
30             temp += (img[i,j] - mid_verdi)**2
31     return temp/(n*m)
32
33     varians = varians_regning(n,m,img, mid_verdi)
34     standard = np.sqrt(varians)
35
36     # Calculate the new a and b
37     a = 64/standard
38     b = 127 - a*mid_verdi
39
40     new_img = img*a +b
41
42     plt.figure()
43     plt.title("Original")
44     plt.imshow(img, cmap='gray', vmin=0, vmax=255)
45
46     plt.figure()
47     plt.title("Endring av kontrast")
48     plt.imshow(new_img, cmap='gray', vmin=0, vmax=255)
49
50     # DEL 2
51     # Standardisere geometrien
52     # Slik at øyne og munn passer over masken
53
54     filename = 'geometrimaske.png'
55     mask = Image.open(filename).convert('L') # 'L' mode ensures the image is
56     ↪ grayscale
57     geometrimaske = np.array(mask)
58     N, M = geometrimaske.shape
59
60     # Now we use this equations to find the constants
61     a_0 = 3.022
62     a_1 = -3.211
63     a_2 = 201.956
64     b_0 = 2.309
65     b_1 = 4.041
66     b_2 = -295.618
67     C = np.array([[a_0, a_1, a_2],
68                   [b_0, b_1, b_2],
69                   [0, 0, 1]])
70
71     def affin_trans(n, m, img, C, N, M):
72         output_image = np.zeros((N,M))
73
74         for i in range(n):
75             for j in range(m):
76                 new_pixel = C @ np.array([j, i, 1])
77                 new_pixel = new_pixel.astype(int)

```

```

77         if 0 <= new_pixel[0] < M and 0 <= new_pixel[1] < N:
78             output_image[new_pixel[1], new_pixel[0]] = img[i,j]
79
80     return output_image
81
82 trans_img = affin_trans(n, m, new_img, C, N, M)
83
84 plt.figure()
85 plt.title("Portrett etter forlengstransformasjon")
86 plt.imshow(trans_img, cmap='gray', vmin=0, vmax=255)
87
88 inv_C = np.linalg.inv(C)
89
90 #Function to compute the neighbor interpolation
91 def nearest_neighbor_interpolation(n, m, img, inv_C, N, M):
92     output_image = np.zeros((N, M))
93
94     for i in range(N):
95         for j in range(M):
96             transformed_coords = inv_C @ np.array([j, i, 1])
97             x_new = round(transformed_coords[0])
98             y_new = round(transformed_coords[1])
99             if 0 <= x_new < m and 0 <= y_new < n:
100                 output_image[i,j] = img[y_new, x_new]
101
102     return output_image
103
104 neigh_trans= nearest_neighbor_interpolation(n, m, new_img, inv_C, N, M)
105
106 plt.figure()
107 plt.title("Portrett etter nabointerpolasjon")
108 plt.imshow(neigh_trans, cmap='gray', vmin=0, vmax=255)
109
110 #Function to compute the bilinear interpolation
111 def bilinear_interpolation(img, inv_C, N, M):
112     output_image = np.zeros((N, M))
113     n, m = img.shape
114
115     for i in range(N):
116         for j in range(M):
117             transformed_coords = inv_C @ np.array([j, i, 1])
118             x = transformed_coords[0]
119             y = transformed_coords[1]
120
121             # Ensure coordinates are within bounds
122             if x < 0 or x >= m - 1 or y < 0 or y >= n - 1:
123                 continue # Skip interpolation for out of bounds points
124
125             # Finding the four neighbors

```

```

127         x_1 = int(np.floor(x))
128         y_1 = int(np.floor(y))
129         x_2 = min(x_1 + 1, m - 1)
130         y_2 = min(y_1 + 1, n - 1)
131
132         # Get the pixel values
133         Q_11 = img[y_1, x_1]
134         Q_21 = img[y_1, x_2]
135         Q_12 = img[y_2, x_1]
136         Q_22 = img[y_2, x_2]
137
138         # Handle possible division by zero (if x_1 == x_2 or y_1 ==
139         ↪ y_2)
140         if x_2 == x_1:
141             x_weight = 0
142         else:
143             x_weight = (x - x_1) / (x_2 - x_1)
144
145         if y_2 == y_1:
146             y_weight = 0
147         else:
148             y_weight = (y - y_1) / (y_2 - y_1)
149
150         # Bilinear interpolation formula
151         top = (1 - x_weight) * Q_11 + x_weight * Q_21
152         bottom = (1 - x_weight) * Q_12 + x_weight * Q_22
153         output_image[i, j] = (1 - y_weight) * top + y_weight * bottom
154
155     return output_image
156
157 bilin_trans = bilinear_interpolation(new_img, inv_C, N, M)
158
159 plt.figure()
160 plt.title("Portrett etter bilineær interpolasjon")
161 plt.imshow(bilin_trans, cmap='gray', vmin=0, vmax=255)
162
163 plt.show()

```

2 Detektering av cellekjerter

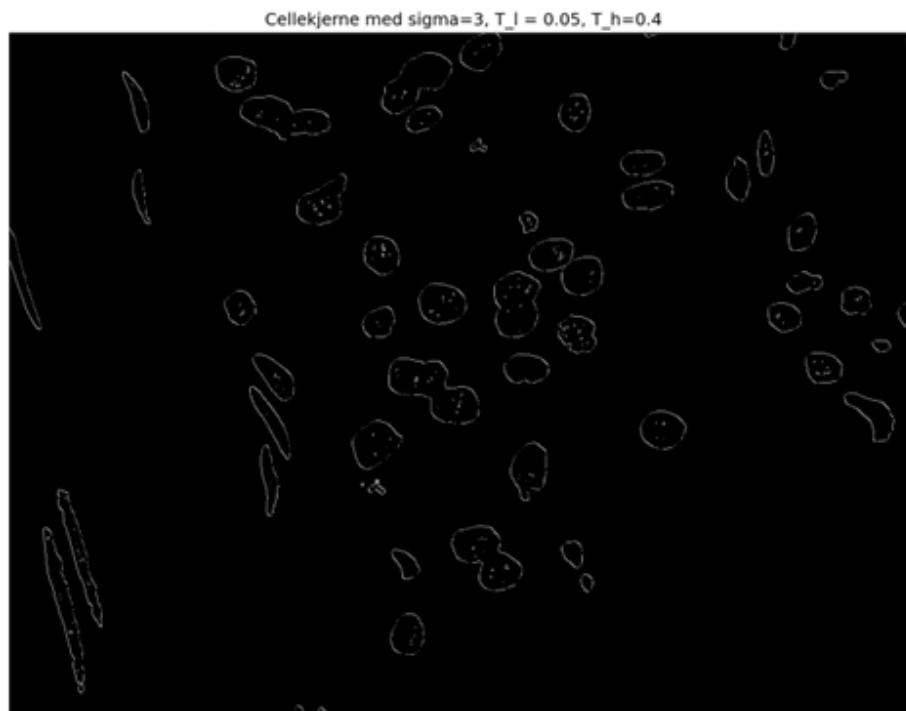
2.1 Forklaring implementering konvolusjonsfilter og Can- nys algoritme.

En konvolusjonsfilter går gjennom hver piksel og summerer alle nabopikler som er i en radius av r . Der r er størrelse av filteret delt på to. Cannys algoritme for kantdeteksjon består av flere steg. Først trenger man å implementere Gauss-filter, dette reduserer støy. Videre beregnes gradientberegning i hver piksel for

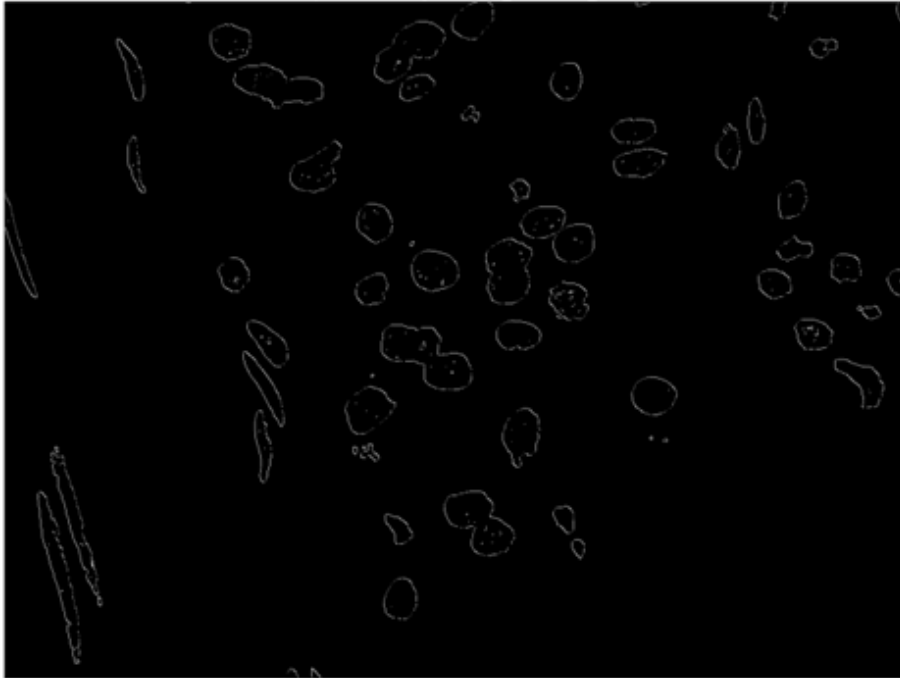
å gi både gradientstyrke og gradientretning for hver piksel. For å sikre at bare de mest signifikante kantene beholdes, undettrykkes pikslene som ikke er lokale maksimum i gradientretningen. Til slutt anvendes husterese-terskling for å fjerne falske kanter, T_l og T_h brukes for å klassifisere kantene som sterke eller svake.

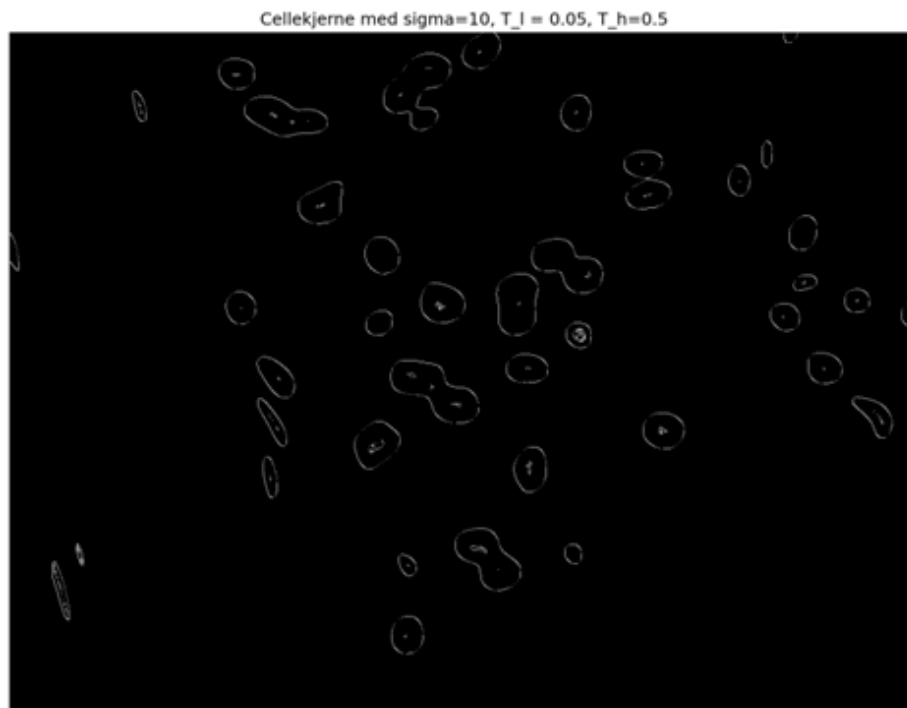
2.2 Resultat bilder

Her er det noen bilderesultat med forskjellige verdier for σ , T_l , og T_h



Cellekjerne med $\sigma=2$, $T_l = 0.1$, $T_h=0.3$





2.3 Drøfting av resultatet

Man kan se at resultatene varierer basert på hvor skarpe kantene i cellene er, hvor mye støy finnes det i bildet, og hvilke parameterverdier som brukes i kantdeteksjonen.

Kantdeteksjonsmetoden fungerer best når kantene er tydelige og kontrastfylte, men den kan slite med diffuse eller overlappende strukturer. Ved å justere sigma og terskelverdiene kan man balansere mellom sensitiviteten til kantene og robustheten mot støy. Dette gir en viss kontroll over hvordan resultatene ser ut, avhengig av hvilken detaljnivå som ønskes i kantene.

2.4 Koden

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 # Load the image
6 filename = 'cellekjerner.png'
7 img = Image.open(filename).convert('L') # 'L' mode ensures the image is
    ↪ grayscale
8 image = np.array(img)
9

```

```

10 def convolve(image, kernel):
11     """Apply convolution of an image with a given kernel."""
12     img_height, img_width = image.shape
13     kernel_height, kernel_width = kernel.shape
14
15     # Padding the image to handle borders
16     pad_height = kernel_height // 2
17     pad_width = kernel_width // 2
18     padded_image = np.pad(image, ((pad_height, pad_height), (pad_width,
19     ↪ pad_width)), mode='edge')
19
20     # Initialize the output
21     output = np.zeros_like(image)
22
23     # Apply convolution
24     for i in range(img_height):
25         for j in range(img_width):
26             # Element-wise multiplication and summing the result
27             neighborhood = padded_image[i:i+kernel_height,
28             ↪ j:j+kernel_width]
29             output[i, j] = np.sum(kernel * neighborhood)
30     return output
31
32 def gaussian_kernel(size, sigma):
33     """Generates a Gaussian kernel."""
34     kernel = np.zeros((size, size))
35     center = size // 2
36     total_sum = 0 # Normalization constant
37
38     for i in range(size):
39         for j in range(size):
40             x = i - center
41             y = j - center
42             value = (1/(2*np.pi*sigma**2)) * np.exp(-(x**2 + y**2) /
43             ↪ (2*sigma**2))
44             kernel[i, j] = value
45             total_sum += value
46
47     # Normalize the kernel so the sum of all elements is 1
48     kernel = kernel / total_sum
49     return kernel
50
51 def sobel_filters(image):
52     """Apply Sobel filters to estimate gradients."""
53     Kx = np.array([[ -1,  0,  1], [-2,  0,  2], [-1,  0,  1]]) # Sobel X
54     Ky = np.array([[ -1, -2, -1], [ 0,  0,  0], [ 1,  2,  1]]) # Sobel Y
55
56     Ix = convolve(image, Kx)
57     Iy = convolve(image, Ky)

```

```

57     G = np.hypot(Ix, Iy) # Magnitude of the gradient
58     theta = np.arctan2(Iy, Ix) # Direction of the gradient
59     return G, theta
60
61 def non_max_suppression(G, theta):
62     """Thin edges by suppressing non-maximum gradient magnitudes."""
63     M, N = G.shape
64     Z = np.zeros((M, N), dtype=np.int32)
65     angle = theta * 180. / np.pi
66     angle[angle < 0] += 180
67
68     for i in range(1, M-1):
69         for j in range(1, N-1):
70             try:
71                 q = 255
72                 r = 255
73
74                 # Angle 0 (horizontal neighbors)
75                 if (0 <= angle[i, j] < 22.5) or (157.5 <= angle[i, j] <=
76                     ↪ 180):
77                     q = G[i, j+1]
78                     r = G[i, j-1]
79                 # Angle 45 (diagonal neighbors - top-right and
80                     ↪ bottom-left)
81                 elif 22.5 <= angle[i, j] < 67.5:
82                     q = G[i+1, j-1]
83                     r = G[i-1, j+1]
84                 # Angle 90 (vertical neighbors)
85                 elif 67.5 <= angle[i, j] < 112.5:
86                     q = G[i+1, j]
87                     r = G[i-1, j]
88                 # Angle 135 (diagonal neighbors - top-left and
89                     ↪ bottom-right)
90                 elif 112.5 <= angle[i, j] < 157.5:
91                     q = G[i-1, j-1]
92                     r = G[i+1, j+1]
93
94                 if (q == 0 or r == 0):
95                     Z[i, j] = G[i, j]
96                 else:
97                     Z[i, j] = 0
98
99             except IndexError as e:
100                 pass
101
102     return Z
103
104 # Thresholding Based on Intensity
105 def intensity_threshold(image, threshold=100):
106     """Keeps only pixels darker than the threshold (darker elements)."""

```

```

104     return np.where(image < threshold, image, 0)
105
106     # Canny Edge Detection Implementation
107     def canny_edge_detection(image, sigma=1, low_threshold=0.1,
108     ↪ high_threshold=0.3, intensity_thresh=100):
109         """Complete Canny Edge Detection algorithm to focus on darker
110         ↪ elements."""
111         # Step 1: Gaussian filter
112         size = int(2*(np.ceil(3*sigma))+1)
113         smoothed_image = convolve(image, gaussian_kernel(size, sigma))
114
115         # Step 2: Apply intensity thresholding to isolate darker elements
116         dark_elements = intensity_threshold(smoothed_image, intensity_thresh)
117
118         # Step 3: Get gradient magnitudes and angles
119         G, theta = sobel_filters(dark_elements)
120
121         # Step 4: Non-maximum suppression
122         non_max_img = non_max_suppression(G, theta)
123
124         # Step 5: Double threshold
125         strong = non_max_img > high_threshold
126         weak = (non_max_img > low_threshold) & (non_max_img <=
127         ↪ high_threshold)
128
129         output = np.zeros(non_max_img.shape)
130         output[strong] = 255
131         output[weak] = 75
132
133         return output
134
135     # Apply the Canny Edge Detection focusing on darker elements
136     edges = canny_edge_detection(image, sigma=2, low_threshold=0.1,
137     ↪ high_threshold=0.3, intensity_thresh=120)
138
139     plt.figure()
140     plt.title("Cellekjerne med sigma=2, T_l = 0.1, T_h=0.3")
141     plt.imshow(edges, cmap='gray', vmin=0, vmax=255)
142     plt.axis("off")
143     plt.show()

```